

Trabalho Final de Complexidade de Algoritmos

Luciano Kohler da Silva¹, Matheus Eduardo Gomes de Santana¹

¹Departamento de Ciência da Computação – Universidade do Estado de Santa Catarina
(Udesc)

R. Paulo Malschitzki, 200 – 89.219-710 – Joinville – SC – Brasil

{luciano.kds, matheus.santana}@edu.udesc.br

Abstract. *This article describes a bidimensional grid traversal problem and presents a solution developed by the authors themselves. Such problem contains a non-polynomial time solution and involves finding an escape route of a “hero” inside a labyrinth featuring “treasures” and “walls”. The problem’s objective is to indicate the possibility of the “hero”’s escapism after collecting all of the labyrinth’s “treasures”. If such solution is possible, the algorithm must also return the solution’s path.*

Resumo. *Este artigo descreve um problema de travessia em um labirinto bidimensional e compartilha uma solução desenvolvida pelos próprios autores para o mesmo. Tal problema possui uma solução em tempo não-polinomial e envolve a escapatória de um “herói” em um labirinto contendo “tesouros” e “paredes”. O objetivo do problema é indicar se é possível que o “herói” saia do labirinto após coletar todos os tesouros. Se é possível encontrar tal saída, o algoritmo também deve retornar o caminho para tal resolução.*

1. Introdução

O problema abordado neste trabalho insere-se nesta categoria de desafios de travessia de grafos (mesmo que o layout do dito grafo seja representado por meio de uma matriz de caracteres). E consiste num labirinto bidimensional estruturado onde um agente principal atravessá-lo de forma à encontrar uma saída válida do labirinto. Apenas essas restrições **não tornariam o problema Não-Polinomial**, então foi criada uma restrição onde a saída só é liberada após a coleta de T tesouros espalhados pelo labirinto. Essa restrição insere um grande problema na implementação do problema, tornando-o não-polinomial pelo uso exacerbado de memória que explode com grandes quantidades de tesouros.

O objetivo primordial deste trabalho é descrever a abordagem algorítmica

desenvolvida pelos próprios autores, avaliar a viabilidade de se alcançar a saída em diferentes configurações de mapas, dividindo essa abordagem em definição do problema, solução proposta, testes realizados, resultados, e uma breve conclusão.

Note que o algoritmo para resolver tal problema está disponível [neste repositório](#)

2. Definição do Problema

Como previamente citado, o problema consiste em um labirinto bidimensional, onde um herói 'A' necessita encontrar uma forma de se locomover até a saída de um labirinto (qualquer borda do labirinto que não é uma parede). Contudo, o labirinto contém 'T' tesouros que o herói deve coletar antes de sair do labirinto. Esse problema não é polinomialmente resolvível pois necessita de um uso de memória massivo, salvando todos os estados possíveis de coleta dos tesouros.

O problema consiste primeiramente em dois inteiros n e m que correspondem, respectivamente, à altura e à largura do labirinto, após isso, são recebidos n linhas com m caracteres indicando o *layout* do labirinto. O mesmo pode (e preferencialmente deve) conter os caracteres:

- 'A': Um caractere único e obrigatório no labirinto que indica a posição do herói no labirinto;
- 'T': Indica a posição de um tesouro no labirinto;
- '.': Indica uma casa vazia e acessível pelo herói;
- '#': Indica uma parede que bloqueia o movimento do herói.

O objetivo específico do problema é o de determinar se é possível encontrar uma rota que o herói possa realizar que resulte na chegada do fim do labirinto **após a coleta de todos os tesouros residentes no labirinto**. Mesmo que o objetivo principal do algoritmo seja apenas o de descobrir um caminho qualquer que resulte na fuga do nosso herói, o algoritmo montado para tal problema também retorna (quando possível), a **mais curta solução** para escapar do labirinto.

Um exemplo de labirinto pode ser compreendido abaixo:

```
#####  
#A...T#  
#.####.#  
#.#...#.#  
#...T.#  
#.....#  
#####.#
```

3. Solução Proposta

Nessa seção, não será colocada a implementação do código devido ao limite de páginas estabelecido pelo professor (5 páginas). Dessa forma, o código será enviado em conjunto com o arquivo PDF deste documento, para que possa ser feita a análise tanto dos comentários contidos neste documento, quanto a análise do código em si. O foi implementado de forma à procurar facilitar a leitura e entendimento do mesmo, utilizando comentários e variáveis nomeadas a partir de sua função.

3.1. Algoritmos Utilizados

Para a resolução do problema, foram utilizadas duas técnicas famosas na área de resolução de problemas:

- **BFS:** Como encontrar o caminho para os tesouros envolve atravessar o labirinto, um algoritmo de travessia de grafos é necessário, a BFS possibilita encontrar o caminho para as chaves com a menor distância possível
- **Bitmask DP:** Para saber quais tesouros já foram coletados, será utilizado uma **bitmask** que armazena os tesouros já coletados, onde **0** significa “não coletado” e **1** significa coletado. Se existem 10 tesouros e foram coletados os tesouros [1, 4 e 8], a bitmask correspondente seria: **1001000100**. Com isso, podemos salvar os estados de cada célula em determinado estado, e se um tesouro foi coletado, podemos apenas acender o bit correspondente.

3.2. Ideia Geral do algoritmo

Com o labirinto recebido, salvamos as coordenadas dos tesouros e do herói e, à partir da coordenada inicial do herói, iniciamos uma BFS que salva como parâmetros:

- A posição X, Y do herói no labirinto
- O estado (bitmask) atual (quantos tesouros foram coletados)

Para cada iteração, se encontramos um tesouro não coletado, inserimos ele à máscara, e se em algum momento chegamos em uma saída com todos os tesouros coletados, encontramos o caminho tomado via *backtracking* e printamos ele na tela. Já se o caminho não foi encontrado (a fila da BFS se esvaziou sem encontrar a solução),

então não há um caminho válido para tal labirinto.

4. Testes realizados e Resultados

Neste tópico, serão inseridos alguns casos teste e seus respectivos resultados para provar sua eficácia, além disso, serão identificados os limites técnicos do problema:

4.1. Testes

Tabela 1. Retornos do algoritmo para diferentes entradas

Entrada	Observação	Saída
7 8 ##### #A...T# #.####.# #.#..#.# #.#.#T.# #.....# #####.#	Labirinto convencional e solucionável	Caminho encontrado com tamanho 12 Caminho: DDDDDBBBEDBB
5 5 ##### #T#.# ###.# #A..# ###.#	Labirinto insolucionável (tesouro isolado)	Labirinto impossível

4.2. Complexidade do algoritmo

Outros testes foram realizados com labirintos de tamanhos e quantidade de tesouros variados. Após devida testagem e teorização matemática, foi concluído que a **complexidade** de memória do algoritmo é dada por $O(N \times M \times 2^T)$, o termo 2^T , responsável por tornar o problema não polinomial, vêm do fato de que, com T tesouros, temos 2^T possibilidades de tesouros diferentes, esse valor ocorre pois este é o tamanho do **conjunto das partes** do conjunto dos tesouros do labirinto, então para representar cada possibilidade do labirinto, precisamos de uma matriz 3D que armazena cada X, Y e estado de tesouros coletados.

Com essa complexidade em mente, podemos inferir os limites do problema

(tendo em mente que o C++ executa, em média, 1×10^8 operações por segundo), e que o algoritmo deve retornar um resultado em até 1 segundo, podemos colocar como limites para N , $M = 100$ e a **quantidade de tesouros** = 10, assim, o algoritmo realizaria em média, $100 \times 100 \times 2^{10}$ operações, totalizando aproximadamente 1×10^7 operações.

5. Conclusão

Após o desenvolvimento e a implementação do algoritmo de resolução do problema, é possível notar que:

- O algoritmo apresenta uma **solução Não-Polinomial** pois o conjunto de estados possíveis de coleta de tesouros explode exponencialmente conforme a quantidade de tesouros no labirinto cresce, criando um problema de memória no algoritmo.
- O uso de **algoritmos de travessia de grafos se mostraram úteis** mesmo quando com o fato do mapa utilizado não apresentar um formato padrão de grafos, isso acontece pois uma grid pode ser interpretada como um grafo onde suas arestas correspondem aos vizinhos dos caracteres
- O sistema foi validado com sucesso através de baterias de casos de teste, operando de forma robusta e **gerando respostas corretas para todos eles**.

6. Referências

SBC (2017). Manual de Formatação de Artigos da SBC. Sociedade Brasileira de Computação.

GeeksForGeeks. Breadth First Search or BFS for a Graph. Disponível: <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>. Janeiro/2026.

GeeksForGeeks. Bitmasking and Dynamic Programming | Travelling Salesman Problem . Disponível: <https://www.geeksforgeeks.org/dsa/bitmasking-dynamic-programming-set-2-tsp/>. Julho/2025